

Lab 3C: MCP — Connecting Claude to External Tools

Duration: 45 minutes

Difficulty: Intermediate to Advanced

Continues from: Lab 3B (dashboard project with commands and hooks)

© 2026 AIA Copilot — Instructor-Led Training Material

Overview

So far, Claude Code can read files, run commands, and write code. MCP (Model Context Protocol) extends Claude's reach to external services — databases, APIs, documentation, and tools that live outside your codebase.

What You'll Build

- Install and configure an MCP server for filesystem operations
 - Connect Claude to a live documentation source
 - Use Chrome DevTools MCP to interact with your running dashboard in the browser
 - Build a custom MCP tool for your dashboard project
 - Understand when to use MCP vs @ mentions vs custom commands
 - Measure the token cost of MCP servers
-

Setup

Continue in your dashboard project:

```
cd business-dashboard
claude
```

Clear Context

```
/clear
```

Phase 1: Understanding MCP

Before we install anything, understand what MCP is:

```
Explain MCP (Model Context Protocol) in one paragraph.  
What problem does it solve that @ mentions and custom commands don't?
```

The short answer: @ mentions give Claude files from your project. Custom commands give Claude instructions. MCP gives Claude **live tools** — the ability to reach out to external systems, query databases, call APIs, and interact with services that aren't part of your codebase.

Phase 2: Install Your First MCP Server

We'll start with the filesystem MCP server — it gives Claude enhanced file operations beyond the built-in tools.

```
Show me how to add an MCP server configuration to this project.  
I want to add the @anthropic/mcp-filesystem server.  
Show the .claude/settings.json configuration needed.
```

After configuring, verify it's working:

```
List the available MCP tools. Do I have filesystem tools available?
```

Try using it:

```
Use the MCP filesystem tools to get detailed metadata about all files  
in the src/routes/ directory — size, modification dates, permissions.
```

Compare this to a regular file listing. MCP provides richer data that the built-in tools don't expose.

Phase 3: Connect to Documentation

MCP really shines when you connect Claude to live documentation. We'll use the fetch MCP server:

```
Add the @anthropic/mcp-fetch server to our MCP configuration.  
This lets Claude fetch URLs and read web content.
```

Now test it with your dashboard project:

```
Fetch the Express.js error handling documentation at  
https://expressjs.com/en/guide/error-handling.html  
and compare our server.js error handling against their best practices.  
What are we missing?
```

This is powerful — Claude is reading live documentation and applying it to your code in real time. No copy-pasting from a browser.

Phase 4: MCP for Database Access

If your dashboard project has SQLite or a database, this is where MCP transforms the workflow:

```
Add the @anthropic/mcp-sqlite server pointing at our project's database file.  
If we don't have a database yet, create a simple SQLite database with our  
dashboard's metrics data.
```

Now query it through Claude:

```
Show me the schema of all tables in the database.  
Then run a query to find the top 5 metrics by value.
```

```
Our metrics endpoint is returning data from a hardcoded array.  
Refactor it to query the actual database instead.  
Use parameterized queries as our CLAUDE.md requires.
```

Claude just went from reading files to querying your database and refactoring code to use it — all in natural language.

Phase 5: Chrome DevTools MCP

This is the MCP server that gets the biggest reaction. Chrome DevTools MCP lets Claude interact with your running application in the browser — clicking buttons, reading rendered content, taking screenshots, and scraping data from pages that don't have APIs.

```
Add the Chrome DevTools MCP server to our configuration.  
Show me the settings.json entry needed.
```

Start your dashboard application:

```
Start the dashboard server in the background (npm start or node server.js)
```

Now use Chrome DevTools through Claude:

```
Open Chrome to http://localhost:3000 and take a screenshot of the dashboard.  
What does the rendered page look like? Are there any visual issues?
```

Try data extraction from the rendered page:

```
Using Chrome DevTools, navigate to the dashboard and extract all visible  
metrics data from the rendered HTML. Compare what's rendered in the browser  
to what the /api/metrics endpoint returns as JSON. Do they match?
```

This is incredibly powerful for: - **Testing**: Verify what users actually see, not just what the API returns - **Scraping**: Extract data from pages that don't have APIs - **Debugging**: See rendered output without leaving your terminal - **Automation**: Click through UI workflows programmatically

Phase 6: When to Use What

This is the decision framework you'll use daily:

```
Based on everything we've done today, create a decision guide as a markdown table:  
When should I use @ mentions vs custom commands vs MCP vs hooks?  
Include: what it does, when to use it, and an example for each.
```

Review Claude's answer against this framework:

Tool	What It Does	When to Use It
@ mentions	Loads project files into context	You know which file to reference
Custom commands	Reusable prompt templates	Repetitive workflows with consistent output
MCP servers	Connects to external tools/ services	Need data from outside the codebase
Hooks	Intercepts and controls Claude's actions	Safety guardrails, logging, policy enforcement

Phase 7: Token Impact Check

MCP servers add tool descriptions to your context. Check the cost:

```
/context
```

Notice how MCP server tool descriptions consume context. A single MCP tool like `update_task` can consume more tokens than ALL your custom skills combined. This is why you only enable the MCP servers you need for the current session. Unused MCP servers waste tokens.

Compare the token cost: - Custom skills: ~50-60 tokens each (only description loaded, not full content) - MCP tools: often 200-500+ tokens each (full schema loaded)

```
How many tokens are our MCP server tools using vs our custom commands?  
Which MCP tools are the most expensive?
```

If an MCP tool is eating tokens but you're not using it this session, disable it.

What You Learned

- MCP extends Claude's reach beyond your local files
- MCP servers are configured in `.claude/settings.json`
- Filesystem, fetch, and database servers cover most needs
- MCP tool descriptions consume context tokens — only enable what you need
- The decision framework: @ mentions for files, commands for workflows, MCP for external tools, hooks for safety

MCP Best Practice

Enable only the MCP servers you need for the current task. Disable the rest. Context is a budget — MCP tools are expensive line items.

Keep your dashboard project open — Lab 3D builds on it.

